

Blender Helper v3.0

Deterministic & Scalable Engine (Автономная экспертная система без ИИ)

🎯 Главная цель проекта

Масштабировать систему поиска, базу знаний и диагностику бота Blender Helper **без использования ИИ, нейросетей и внешних API**. Все алгоритмы должны быть детерминированными, работать на 100% локально на бесплатном сервере Oracle Cloud и гарантировать точное отсутствие галлюцинаций (Zero-Hallucination Mode).



Модуль 1. Улучшение локального алгоритмического поиска

1.1. Алгоритм BM25 (замена базовому TF-IDF)

- **Переход на BM25 (Окари BM25):** Реализовать или подключить локальный легкий алгоритм BM25 на чистом Python (без тяжелых внешних библиотек). BM25 значительно лучше учитывает длину документа и насыщение ключевыми словами, чем стандартный TF-IDF.
- **Усиленный лемматизатор и словари синонимов:**
 - Расширить словарь двуязычной терминологии (`knowledge/system/terminology`).
 - Добавить автоматическую нормализацию пользовательских опечаток с помощью алгоритма расстояния Левенштейна (fuzzy matching через `rapidfuzz` или `difflib`). Если пользователь пишет с ошибкой («бевел», «модификатр»), бот всё равно корректно нормализует слово.

1.2. Иерархическое ранжирование и комбинация запросов

Учитывать жесткие приоритеты источников при формировании ответа:

1. Точное совпадение по словарю синонимов / горячих клавиш.
2. Личные проверенные заметки (`knowledge/personal/`).
3. Официальный Blender Manual (`knowledge/official/`).

Порог уверенности (Confidence Threshold): Если итоговый балл релевантности ниже T_{min} , бот не выдает случайный текст, а честно предлагает уточняющие кнопки или список похожих тем.



Модуль 2. Автоматическое масштабирование Базы Знаний

2.1. Полноценный парсер Blender Manual (`scripts/parse_manual.py`)

- Написать скрипт-парсер, который автоматически скачивает и структурирует актуальную документацию Blender (HTML/RST).
- **Умное чанкирование:** Разбивать статьи не по фиксированному количеству символов, а по логическим блокам HTML:
 - Заголовок + абзац описания.
 - Блоки `Note` , `Warning` и таблицы параметров.
- **Метаданные:** На каждый чанк автоматически навешивать теги: `category` , `blender_version` , `source_url` .

2.2. Импорт и структурирование хоткеев

- Написать скрипт конвертации официальных таблиц горячих клавиш Blender в формат `data/hotkeys.json` для моментального ответа по категориям.

🔧 Модуль 3. Пошаговые алгоритмические сервисы

3.1. Расширение диагностических деревьев (Decision Trees)

- Пополнить `knowledge/system/diagnostics/` новыми ветками сценариев:
 - Проблемы с UV-разверткой (потяги текстур, наложение UV-островов).
 - Проблемы с запеканием карт (Baking artifacts / cage issues).
 - Проблемы с симуляциями (ткань, физика или жидкость не реагируют на коллизии).
- Добавить поддержку отправки наглядных скриншотов-схем на ключевых шагах диагностики.

3.2. Панель администрирования через Telegram

- **Команда `/unanswered`** : Отображение списка реальных вопросов пользователей, на которые поиск дал оценку `confidence < threshold` .
- **Команда `/quick_add`** : Возможность прямо из Telegram вбить ответ на неизвестный вопрос. Скрипт сам формирует JSON-файл в `knowledge/personal/` , обновляет индекс и делает `/reindex` без перезапуска сервиса.

🧪 Модуль 4. Тестирование и авто-валидация

4.1. Расширение Unit-тестов и Quality Suite

- Написать тесты на проверку нормализации слов, устойчивость к опечаткам и корректность обработки синонимов.
- Создать автоматический тест, проверяющий, что время отклика на любой запрос не превышает 50 миллисекунд.

Документ сформирован автоматически для передачи в Claude Code • Проект Blender Helper